IN1007 · PARADIGMAS DE LINGUAGENS DE PROGRAMAÇÃO · 2026.2

Enumerações e União de Tipos na Linguagem Funcional 1

Uma extensão da LF1 que mantém a sintaxe original e a inferência de tipos

Luiz Primo · Kennedy Melo

Centro de Informática — UFPE

Prof. Augusto Sampaio

CONTEXTO

Na LF1, toda expressão tem exatamente um tipo

A LF1 é a linguagem funcional da disciplina: funções recursivas, declarações com **let** e tipos **inferidos**, sem anotações.

Ela conhece apenas três tipos:

`Inteiro` `Booleano` `String`

E os dois ramos de um **if** precisam ter o mesmo tipo.

EXEMPLO

```
if ok then 42 else "erro"
```

⚠ Rejeitado pela LF1: um ramo é Inteiro e o outro é String

MOTIVAÇÃO

Duas situações comuns que a LF1 não expressa

PROBLEMA 1

Alternativas fechadas

Cores, dias da semana, estados de um pedido. Hoje viram números ou strings “mágicos”.

Proposta: enumerações (**enum**)

PROBLEMA 2

Tipos diferentes

Uma expressão que, conforme a condição, produz um Inteiro ou uma String.

Proposta: união de tipos

> Nos dois casos, a regra é a mesma: **segurança de tipos**. Nenhuma operação pode ser aplicada a um valor que ela não suporta.

EXTENSÃO 1 · ENUMERAÇÕES

Um conjunto fechado de constantes com nome

```
let enum Cor = Vermelho | Verde | Azul in ...
```

Novo tipo

Cor passa a ser um tipo, válido no corpo do let.

Constantes

Verde é um identificador comum, do tipo Cor.

Nominal

Enums diferentes nunca se misturam. Compara-se com ==.

EXTENSÃO 2 · UNIÃO DE TIPOS

A união é inferida, nunca escrita

```
if ok then 42 else "erro"
-- tipo: Inteiro | String
```

NORMALIZAÇÃO

Ordem e repetição não importam. Uniões aninhadas são achatadas.

EXECUÇÃO

Nada de encapsulamento: o valor é só um inteiro ou uma string.

EXTENSÃO 2 · UNIÃO DE TIPOS

Como se obtém um valor de tipo união

COM VAR

```
let var v =
  if (1 == 1) then 42 else "erro"
-- v : Inteiro | String
```

COM FUN

```
let fun converte n =
  if (n == 0) then "zero" else n
-- retorno : Inteiro | String
```

O tipo não cresce depois da declaração. Sem atribuição, o tipo de v fica fixo. Um let interno pode criar outro v, que esconde o externo:

```
let var v = if (1 == 1) then 42 else "erro"    -- Inteiro | String
in let var v = if (1 == 2) then v else true    -- Inteiro | String | Booleano
```

EXTENSÃO 1 · ENUMERAÇÕES

Um conjunto fechado de constantes com nome

```
let enum Cor = Vermelho | Verde | Azul in ...
```

Novo tipo

Cor passa a ser um tipo, válido no corpo do let.

Constantes

Verde é um identificador comum, do tipo Cor.

Nominal

Enums diferentes nunca se misturam. Compara-se com ==.

> Um enum é a união mais simples: {Vermelho} | {Verde} | {Azul}. Por isso ele é o ponto de partida.

EXTENSÃO 2 · UNIÃO DE TIPOS

A união é inferida, nunca escrita

```
if ok then 42 else "erro"
-- tipo: Inteiro | String
```

FORMAÇÃO

Ramos de tipos diferentes formam uma união. Ramos iguais: como na LF1.

NORMALIZAÇÃO

Ordem e repetição não importam. Uniões aninhadas são achatadas.

USO

+, length, ++ e == não aceitam uniões. Antes, é preciso testar o tipo.

EXECUÇÃO

Nada de encapsulamento: o valor é só um inteiro ou uma string.

EXTENSÃO 2 · IS E DESESTRUTURAÇÃO

Testar o tipo e depois usar com segurança

```
let var v = if (1 == 1) then 42 else "erro"
in if v is Inteiro then v + 1 else 0    -- resultado: 43
```

INTROSPECÇÃO

`e is T`

Sempre Booleano. Na execução, consulta o tipo do valor de e.

DESESTRUTURAÇÃO

`if x is T then … else …`

No then, x tem tipo T. No else, os demais tipos da união.

Vale só para a forma x is T usada direto como condição. Algo como not (x is T) é válido, mas não estreita o tipo de x.

BNF ESTENDIDA

Só acrescentamos: nenhuma regra da LF1 muda

```
Expressao ::= Valor | ExpUnaria | ExpBinaria
            | ExpDeclaracao | Id | Aplicacao
            | IfThenElse
            | ExpChecagemTipo (+)

DeclaracaoFuncional ::= DecVariavel | DecFuncao
| DecComposta
| DecEnum (+)

DecEnum ::= "enum" Id "=" ListConstEnum (+)
ListConstEnum ::= Id | Id "|" ListConstEnum (+)
ExpChecagemTipo ::= Expressao "is" Tipo (+)
Tipo ::= "Inteiro" | "Booleano" | "String" | Id (+)
```

2
alternativas novas em produções que já existiam

4
produções novas

0
regras da LF1 alteradas ou removidas

A união de tipos não aparece na gramática: ela surge da regra de tipos do if.

EXEMPLOS

Enumeração com análise de casos

```
let enum Semaforo = Verde | Amarelo | Vermelho
in let fun acao s =
      if (s == Verde) then "siga"
      else if (s == Amarelo) then "atencao"
      else "pare"
   in acao(Amarelo)
```

RESULTADO

"atencao"

O tipo de s é inferido como Semaforo, a partir das comparações com as constantes.

O último else cobre a constante que sobrou: Vermelho.

EXEMPLOS

União de três tipos: cada else retira um

```
Let var x = let var v = if (1 == 1) then 10 else if (1 == 2) then "abc" else true
in if v is Inteiro then v + 1
   else if v is String then v
   else if v then v else v
```

Ponto do programa	Tipo de v
Primeiro then	Inteiro
Primeiro else	String | Booleano
Segundo then	String
Segundo else	Booleano

RESULTADO

11

No fim, sobra só Booleano: v pode ser usado direto como condição.

EXEMPLOS

O que o verificador de tipos rejeita

```
let var v = if (1 == 1) then 1 else "a"
in v + 1
```

ERRO DE TIPO

v pode ser String, e + exige dois inteiros.

```
let enum Cor = Vermelho | Verde, enum Fruta = Banana |
Maca in Vermelho == Banana
```

ERRO DE TIPO

Cor e Fruta são enums diferentes: não se comparam.

```
let var v = if (1 == 1) then 1 else "a"
in if (not (v is Inteiro)) then length v else 0
```

ERRO DE TIPO

A condição não é x is T, então v não é estreitado e length v é rejeitado.

PRÓXIMOS PASSOS

Implementação no módulo Funcional1

A proposta está especificada. A implementação segue os pacotes que o módulo já tem, em três etapas:

1

Enumerações

`TipoEnum · ValorEnum`
`DecEnum`

Tipo, valor e declaração.

Igualdade em ExpEquals.

→

2

Introspecção

`ExpChecagemTipo`
`Functional1.jj`

Expressão e is T e os novos tokens no parser.

→

3

Uniões

`TipoUniao`
`IfThenElse`

Formação de uniões e estreitamento no if.

Também: o AmbienteFuncional passa a registrar as enumerações visíveis em cada escopo.

Centro de Informática UFPE

Universidade Federal de Pernambuco

Obrigado

Perguntas?

Luiz Primo · Kennedy Melo

lvps@cin.ufpe.br · kecm@cin.ufpe.br